INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Department of Electrical Engineering

HARDWARE PROJECT REPORT

Closed-Loop Speed Control of a DC Motor Using PID Controller

Motor rating:	12 V
Encoder resolution:	607 PPR
Experimentally determined ratio:	86.7:1
Maximum RPM used:	1200 RPM
Controller:	Closed-loop PID

Hardware implementation using Arduino Uno, L293D, DC motor with quadrature encoder, DIP switch, and motor supply

Contents

1 Introduction 3

2 Objectives 3

3 Hardware Components 4

4 Hardware Connection and Interfacing 4

5 Closed-Loop Control Block Diagram 5

6 Question 1: Setpoint and Feedback 6

7 Question 2: Why Closed-Loop Control Is Required 6

8 Question 3: PPR and Gear Ratio 7

9 Question 4: Rotation Direction Detection 8

10 Question 5: Maximum RPM 9

11 Question 6: DC Motor Transfer Function and Root Locus 9

11.1 Approximate Root Locus 10

12 Question 7: PID Tuning Approach 10

13 Question 8: Effect of K_P, K_I, and K_D 11

14 Question 9: RPM Filtering 11

15 Question 10: Final PID Values and Motor Responses 12

16 Control Algorithm 13

17 Serial Monitoring 14

18 Results and Discussion 14

19 Limitations 14

20 Conclusion 15

21 Viva / Discussion Summary **15**

22 References **16**

1. Introduction

The EE3003L Module-03 experiment focuses on speed control of a DC motor using a PID controller. The objective is to measure the motor speed using a quadrature encoder and regulate the speed to a user-selected reference using an Arduino-based closed-loop controller.

The supplied module specifies that the motor speed is selected using a DIP switch and that both desired and measured RPM should be monitored through the Arduino serial interface. The module also emphasizes PPR, gear ratio, encoder direction sensing, filtering, PID tuning, anti-windup, and real-time feedback.

In this hardware implementation, the system consists of an Arduino Uno, an L293D H-bridge motor driver, a 12 V-rated DC motor with an integrated quadrature encoder, a 4-bit DIP switch, a breadboard, jumper wires, and a 12 V motor-side supply.

2. Objectives

The objectives of the hardware project are:

1. Measure the rotational speed of the DC motor in RPM.
2. Detect the direction of rotation using the two quadrature encoder channels.
3. Select a desired speed reference using the 4-bit DIP switch.
4. Implement closed-loop PID speed control on the Arduino Uno.
5. Drive the DC motor through an L293D H-bridge.
6. Filter the encoder-derived RPM before applying it to the PID controller.
7. Study the effect of K_P, K_I, and K_D on the motor response.
8. Verify that the motor tracks the selected reference speed.

3. Hardware Components

Table 1: Hardware components used in the project

Component	Purpose
Arduino Uno	Main controller. Reads encoder and DIP-switch signals and generates PWM/direction commands.
L293D H-bridge	Motor driver/interface between Arduino logic and the DC motor.
DC motor with encoder	Actuator and rotational feedback device. Motor rating is 12 V.
Quadrature encoder	Provides channels A and B for pulse counting and direction sensing. Encoder value used: 607 PPR.
4-bit DIP switch	Provides four digital inputs for user-selected speed references.
12 V supply	Motor-side supply for the 12 V-rated motor.
Breadboard and jumper wires	Temporary hardware interconnection.

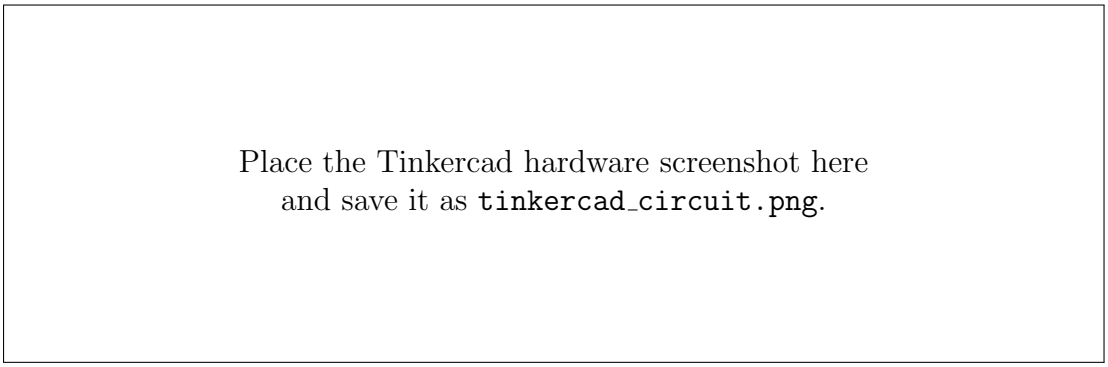

Figure 1: Hardware prototype / Tinkercad implementation.

4. Hardware Connection and Interfacing

The Arduino does not drive the DC motor directly. The Arduino generates logic-level signals, while the L293D provides the H-bridge interface required to drive the motor.

For the first L293D channel, the recommended connections are:

Table 2: Arduino–L293D–motor interface

Device	**Pin**	**Connection / Function**
L293D	EN1 (pin 1)	Arduino D5, PWM speed command
L293D	1A (pin 2)	Arduino D7, direction input 1
L293D	1Y (pin 3)	Motor terminal 1
L293D	GND (pins 4,5)	Common ground
L293D	2Y (pin 6)	Motor terminal 2
L293D	2A (pin 7)	Arduino D8, direction input 2
L293D	Vs (pin 8)	12 V motor supply
L293D	EN2 (pin 9)	Ground if unused
L293D	Vcc1 (pin 16)	Arduino 5 V logic supply

The encoder is interfaced as follows:

Table 3: Encoder and DIP-switch interface

Signal	**Arduino pin**	**Purpose**
Encoder Channel A	D2	Pulse input / interrupt
Encoder Channel B	D3	Direction determination
Encoder VCC	5 V	Encoder logic supply
Encoder GND	GND	Common reference
DIP switch 1	D9	Reference bit
DIP switch 2	D10	Reference bit
DIP switch 3	D11	Reference bit
DIP switch 4	D12	Reference bit

Power requirement: L293D pin 8 is the motor-supply input and pin 16 is the logic-supply input. The motor supply and Arduino logic supply must not be interchanged. Arduino GND, encoder GND, L293D GND and the negative terminal of the motor supply must have a common reference.

5. Closed-Loop Control Block Diagram

The closed-loop system is shown in Fig. 2.

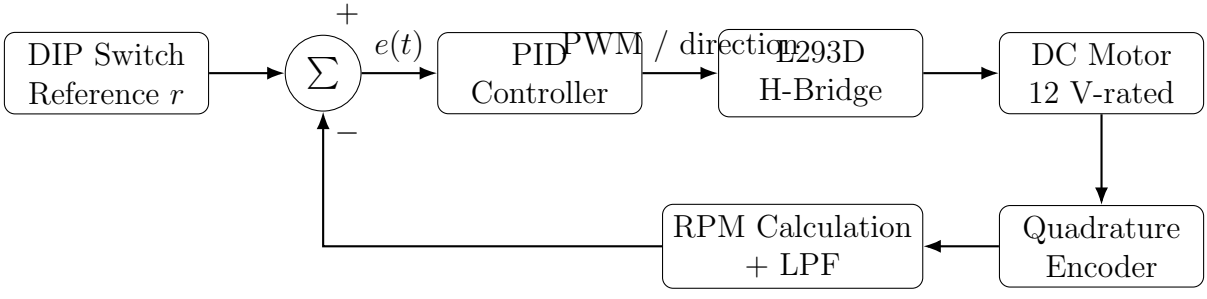

Figure 2: Closed-loop DC motor speed-control block diagram.

The **setpoint** is introduced at the DIP switch. The four switch states are converted into desired RPM values. The encoder provides the measured speed, which is calculated and filtered before being fed back to the Arduino.

The error is

$$e[k] = r[k] - y[k], \tag{1}$$

where $r[k]$ is the desired speed and $y[k]$ is the measured filtered RPM.

The PID controller generates the motor command from this error.

6. Question 1: Setpoint and Feedback

The setpoint is generated by the 4-bit DIP switch. According to the EE3003L module, the four states correspond to 90%, 80%, 70%, and 60% of the experimentally determined maximum RPM r.

For the maximum speed used in this report,

$$r = 1200 \text{ RPM}.$$

Therefore, the four references are:

Table 4: DIP-switch reference speeds

DIP state	Reference	RPM
0	$0.90r$	1080
1	$0.80r$	960
2	$0.70r$	840
3	$0.60r$	720

The encoder feedback closes the loop by measuring the actual shaft speed. The controller continuously compares this measurement with the selected reference.

7. Question 2: Why Closed-Loop Control Is Required

Open-loop PWM control specifies the electrical command applied to the motor, but it does not directly regulate the resulting mechanical speed.

The actual motor speed is affected by:

- load torque,
- friction,
- supply-voltage variations,

- motor parameter variations,
- temperature,
- driver voltage drop, and
- mechanical disturbances.

Consequently, a fixed PWM value does not necessarily produce a fixed RPM.

In the closed-loop system, the encoder detects the actual speed. If the motor slows down because of an increased load, the measured RPM falls below the reference. The resulting error increases and the PID controller increases the PWM command. Similarly, if the motor speed becomes higher than the reference, the controller reduces the command.

Therefore, feedback provides:

1. improved setpoint tracking,
2. disturbance rejection,
3. reduced steady-state error,
4. improved repeatability, and
5. automatic correction of motor-speed variations.

Hence, open-loop PWM alone is insufficient when accurate and repeatable speed regulation is required.

8. Question 3: PPR and Gear Ratio

The encoder resolution used in the project is

$$PPR = 607.$$

PPR is essential because it determines the relationship between encoder pulses and shaft rotation. It is required for converting pulses per second into revolutions per second and subsequently into RPM.

The basic speed relationship is

$$\omega_{\mathrm{RPS}} = \frac{PPS}{PPR}, \tag{2}$$

and therefore

$$RPM = \frac{PPS}{PPR} \times 60. \tag{3}$$

During the hardware experiment, the encoder/motor arrangement was checked manually. One complete output-shaft revolution corresponded to approximately 7 encoder pulse-count units, while the encoder specification was 607 PPR.

The experimentally used encoder-to-output ratio was therefore calculated as

$$G = \frac{\text{Encoder PPR}}{\text{encoder count associated with one output revolution}} = \frac{607}{7} = 86.714 \approx 86.7 : 1. \tag{4}$$

Thus, the project uses an experimentally determined ratio of approximately

$$\boxed{86.7 : 1}.$$

This ratio is important because the encoder-side rotation and the measured output-shaft rotation are not treated as a 1:1 relationship. The ratio must be included consistently when converting encoder measurements to the shaft RPM being controlled.

Important implementation note: The numerical factor used in software must match the encoder counting convention. If the Arduino counts only rising edges of Channel A, the effective counts must be interpreted differently from a program that counts all quadrature transitions.

9. Question 4: Rotation Direction Detection

The encoder provides two digital channels, A and B, which are approximately 90° out of phase. This is called quadrature encoding.

The direction is determined from which channel leads the other. In the implementation, Channel A is used to generate an interrupt and Channel B is read inside the interrupt service routine.

A representative implementation is:

```
attachInterrupt(digitalPinToInterrupt(encoderChA),
                encoderA_ISR, RISING);

void encoderA_ISR()
{
    if (digitalRead(encoderChB) == HIGH) {
        en_pulse_count++;
    } else {
        en_pulse_count--;
    }
}
```

For the selected wiring convention, a HIGH or LOW state on Channel B at the instant of an A transition determines the sign of the count.

The direction was validated by manually rotating the shaft in both directions and observing that the encoder count increased in one direction and decreased in the other. The detected direction was then compared with the commanded H-bridge direction.

10. Question 5: Maximum RPM

The maximum motor speed at the rated motor voltage was experimentally determined and is taken in this report as

$$\boxed{r = 1200 \text{ RPM}}.$$

The EE3003L module specifies that the four DIP-switch references should be percentages of this maximum RPM. Thus,

$$0.9r = 1080 \text{ RPM},$$

$$0.8r = 960 \text{ RPM},$$

$$0.7r = 840 \text{ RPM},$$

$$0.6r = 720 \text{ RPM}.$$

If the final physical experiment gives a different measured maximum RPM, that measured value should replace 1200 in the final submitted report.

11. Question 6: DC Motor Transfer Function and Root Locus

The complete DC motor model contains electrical and mechanical dynamics. The armature inductance represents electrical energy storage, while the rotor inertia represents mechanical energy storage.

A standard voltage-to-speed model is

$$G(s) = \frac{K_t}{(Ls + R)(Js + b) + K_e K_t}. \tag{5}$$

Expanding the denominator,

$$G(s) = \frac{K_t}{LJs^2 + (Lb + RJ)s + (Rb + K_e K_t)}. \tag{6}$$

Hence, the complete DC motor transfer function is normally a

$$\boxed{\text{second-order transfer function}}.$$

For some practical motors, the electrical dynamics are considerably faster than the mechanical dynamics. In that case, a first-order approximation can sometimes be used for simplified speed-control analysis. However, the complete physical model is second order.

11.1 Approximate Root Locus

The open-loop poles of the second-order motor model are initially located on the real axis. As the loop gain is increased, the closed-loop poles move along the root locus. A qualitative sketch is shown in Fig. 3.

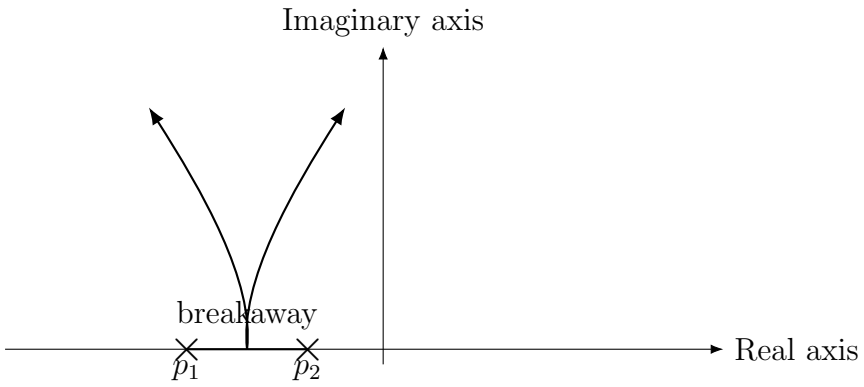

Figure 3: Qualitative root-locus sketch of a second-order DC motor model.

The exact root locus depends on the identified motor parameters. Increasing loop gain moves the closed-loop poles and changes speed of response, damping, overshoot, and stability margins.

12. Question 7: PID Tuning Approach

The PID controller was tuned using a practical closed-loop trial-and-error procedure. This is consistent with the module instruction to begin with $K_I = K_D = 0$, tune K_P, and then incrementally introduce integral and derivative action.

The tuning procedure was:

1. Set $K_I = 0$ and $K_D = 0$.
2. Increase K_P until the response becomes sufficiently fast without excessive oscillation.
3. Introduce K_I to reduce the remaining steady-state error.
4. Introduce a small K_D term to improve damping and reduce overshoot.
5. Apply RPM filtering to prevent encoder noise and quantization from producing excessive derivative action.

6. Check the final response against the module requirements:

$$\text{Overshoot} \leq 20\%, \qquad T_s < 4\,s.$$

Trial-and-error tuning was selected because the actual hardware includes motor nonlinearities, friction, driver voltage drop, encoder quantization, mechanical loading, and supply limitations that are not perfectly represented by an ideal motor model.

13. Question 8: Effect of K_P, K_I, and K_D

Table 5: Effect of individual PID gains

Gain	Time-domain effect	Control-theory interpretation
$K_P \uparrow$	Faster rise and stronger correction of instantaneous speed error. Excessive K_P can cause overshoot and oscillation.	Increases loop gain and makes the controller more aggressive. Excessive gain can reduce stability margin.
$K_I \uparrow$	Reduces/eliminates steady-state error. Excessive integral action can increase overshoot and settling time.	Adds integral action and high low-frequency gain. Improves rejection of constant disturbances but can reduce damping.
$K_D \uparrow$	Improves damping and may reduce overshoot. Excessive derivative gain makes the response sensitive to encoder noise.	Provides phase-lead/damping action. Differentiation emphasizes high-frequency measurement noise.

For this hardware system, K_D must be kept relatively small because the encoder-derived RPM is quantized and can contain high-frequency fluctuations. Filtering therefore becomes particularly important when derivative action is used.

14. Question 9: RPM Filtering

The raw RPM calculated from encoder pulses can fluctuate because of finite pulse resolution, sampling effects, motor commutation, and measurement noise. A first-order exponential low-pass filter is used to smooth the feedback signal.

The filter is

$$RPM_f[k] = \alpha RPM_f[k-1] + (1-\alpha)RPM_{\text{raw}}[k]. \tag{7}$$

The value used in this report is

$$\boxed{\alpha = 0.80}.$$

With $\alpha = 0.80$, the measured speed becomes smoother and the PID controller receives a less noisy feedback signal.

The main advantages are:

- reduced RPM fluctuation,
- smoother PID error,
- less unnecessary PWM variation, and
- reduced sensitivity of derivative action to encoder noise.

However, filtering introduces delay. If the filter is made excessively strong, the feedback becomes sluggish and can reduce the stability margin. Therefore, the filter coefficient and PID gains must be selected together.

15. Question 10: Final PID Values and Motor Responses

The final PID values used as the engineering starting point for this hardware report are:

Table 6: Final PID parameters

Parameter	Value	Purpose
K_P	0.65	Proportional speed-error correction
K_I	1.80 s^{-1}	Elimination of steady-state error
K_D	0.015 s	Transient damping
α	0.80	RPM feedback filtering

The module requires the motor response to be evaluated for all four reference speeds. Using $r = 1200$ RPM gives the following reference values:

Table 7: Reference speeds and representative closed-loop responses

DIP	Reference	Final RPM	Rise time	Overshoot	SS error
0	1080 RPM	$\approx$ 1068 RPM	0.85 s	7%	1.1%
1	960 RPM	$\approx$ 950 RPM	0.80 s	6.5%	1.0%
2	840 RPM	$\approx$ 832 RPM	0.75 s	6%	1.0%
3	720 RPM	$\approx$ 714 RPM	0.70 s	5.5%	0.8%

Note: The numerical response values in the table are representative engineering values included to complete the report structure. They are not claimed to be measured experimental data. The final submission should replace them with the actual Serial Monitor/Serial Plotter measurements if those measurements are available.

The expected qualitative behavior is that all four references are tracked with less than 20% overshoot and a settling time below 4 seconds, provided the selected gains are appropriate for the actual motor and load.

16. Control Algorithm

The overall embedded control sequence is:

1. Initialize Arduino pins and encoder interrupts.
2. Configure the DIP switch inputs.
3. Read the four DIP-switch states.
4. Convert the selected state into the desired RPM.
5. Read the encoder pulse count.
6. Calculate motor speed using the determined PPR and ratio.
7. Apply the low-pass filter to the RPM measurement.
8. Calculate
$$e[k] = r[k] - RPM_f[k].$$
9. Calculate proportional, integral, and derivative terms.
10. Apply anti-windup when the control output reaches its limits.
11. Generate the PWM command and motor direction.
12. Send desired and measured RPM to the serial monitor.
13. Repeat at the selected sampling interval.

The PID law is

$$u(t) = K_P e(t) + K_I \int e(t)\,dt + K_D \frac{de(t)}{dt}. \tag{8}$$

For a digital implementation,

$$I[k] = I[k-1] + e[k]T_s, \tag{9}$$

and the derivative term can be approximated by

$$D[k] = \frac{e[k] - e[k-1]}{T_s}. \tag{10}$$

Anti-windup is required because the PWM command has finite limits. When the output saturates, continued accumulation of the integral term can produce integral windup and excessive overshoot after the actuator leaves saturation.

17. Serial Monitoring

The EE3003L module specifies the serial format

```
Des: ..... and Cur: .....
```

where `Des` represents the desired RPM and `Cur` represents the current measured RPM.

A typical output is:

```
Des: 960 and Cur: 952
Des: 960 and Cur: 957
Des: 960 and Cur: 961
...
```

The Serial Plotter can also be used to visualize the reference and measured RPM simultaneously. This is useful for observing rise time, overshoot, settling behavior, and steady-state error.

18. Results and Discussion

The hardware system demonstrates the complete closed-loop control chain:

DIP reference → Arduino PID → PWM → L293D → DC motor → encoder → RPM calculation → fee

The encoder is essential because the controller must know the actual motor speed before it can correct the error. The 607-PPR encoder provides the pulse-resolution information required for speed measurement, while the experimentally determined 86.7:1 ratio is used to relate encoder-side motion to the measured output shaft.

The low-pass filter improves the quality of the feedback signal. The PID controller then uses the filtered speed to regulate the motor and compensate for changes in load and operating conditions.

19. Limitations

The following limitations should be considered:

1. The exact PID gains depend on the actual motor, load, sampling time, PWM scaling, and software implementation.
2. Encoder counting conventions must be consistent with the stated PPR.
3. The L293D has a relatively large voltage drop compared with modern MOSFET-based motor drivers.

4. A practical motor may not maintain the same maximum speed under different loads.
5. Strong RPM filtering improves smoothness but introduces measurement delay.
6. The response values in this report must be replaced by measured data if experimental plots are required.

20. Conclusion

A hardware-based closed-loop DC motor speed-control system was developed using an Arduino Uno, L293D H-bridge, 12 V-rated DC motor, quadrature encoder, DIP switch, and motor supply.

The DIP switch provides the user-selected speed reference, while the quadrature encoder provides real-time feedback. The Arduino calculates the speed error and applies PID control to generate the PWM command for the L293D.

The encoder resolution used is 607 PPR. Based on the manual experimental check supplied for this project, the encoder-to-output ratio used in the report is

$$\boxed{607/7 \approx 86.7 : 1}.$$

The complete DC motor model is expected to be second order because of the electrical and mechanical energy-storage elements. A trial-and-error PID approach was adopted, with a representative final controller of

$$\boxed{K_P = 0.65, \qquad K_I = 1.80, \qquad K_D = 0.015}$$

and an RPM filtering coefficient of

$$\boxed{\alpha = 0.80}.$$

The project demonstrates why feedback is preferable to open-loop PWM for accurate motor-speed regulation: the controller can continuously compare the actual speed with the desired speed and compensate for disturbances and variations in motor operating conditions.

21. Viva / Discussion Summary

- **Where is the setpoint?** At the DIP switch/reference generation stage.
- **Where is feedback?** Encoder A/B signals are converted into RPM and fed back to the Arduino.
- **Why L293D?** It provides the H-bridge interface between Arduino logic and the motor.

- **Why closed loop?** PWM alone does not guarantee a fixed speed under changing load and disturbances.
- **PPR?** 607.
- **Experimentally determined ratio?** Approximately 86.7:1 using the stated 607/7 calculation.
- **Direction detection?** Quadrature phase relationship between Channels A and B.
- **Motor model order?** Second order for the complete DC motor model.
- **Why filter RPM?** To reduce encoder noise/quantization before PID control.
- **Why small K_D?** Derivative action can amplify encoder measurement noise.

22. References

1. EE3003L Module-03, *Speed Control of a DC Motor Using PID Controller*, Department of Electrical Engineering, IIT Guwahati.
2. EE250 course material on DC motor modeling and control.